

Directions in Advanced Authenticated Encryption

Kazuhiko Minematsu (NEC)

Generic Attacks and Proofs in Symmetric-key cryptography (GAPS) 2026

21 September 2026, Vienna, Austria

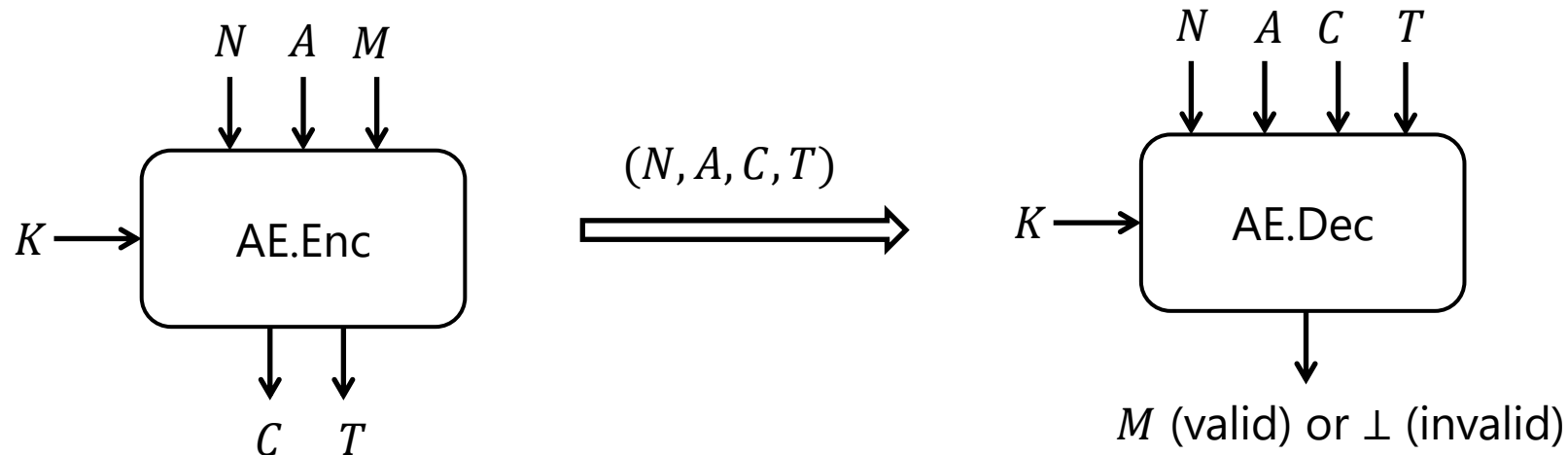
Introduction

- ◆ Authenticated Encryption (AE) becoming nearly “encryption” in digital world
- ◆ Lots of advancement for efficiency and security in the established framework [BN00] [Rog02] [RS06]
 - confidentiality and authenticity
 - robustness (e.g. nonce misuse)
- ◆ But it's not the end of the story: I select two topics on AE with advanced security features and talk about some of recent works
- ◆ Leakage-resilient AE (LRAE)
- ◆ Committing AE (comAE)

Leakage-resilient Authenticated Encryption

(Plain) Authenticated Encryption, AE

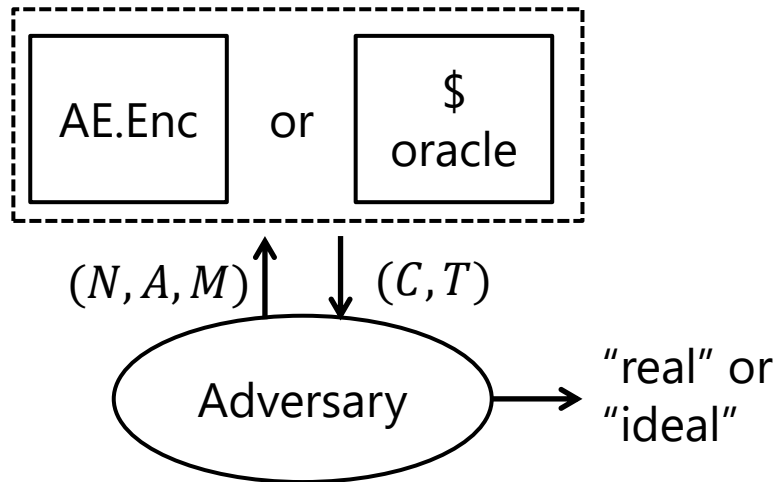
- ◆ Simultaneous encryption and authentication at one scheme
 - Key K , Nonce N , Associated data (AD) A , Plaintext M , Ciphertext C , and Tag T
 - AD is authenticated but not encrypted, $|T|$ is usually fixed & small
- ◆ Nonce N must be unique (but chosen) in encryption
- ◆ Decryption accepts any (repeating) N



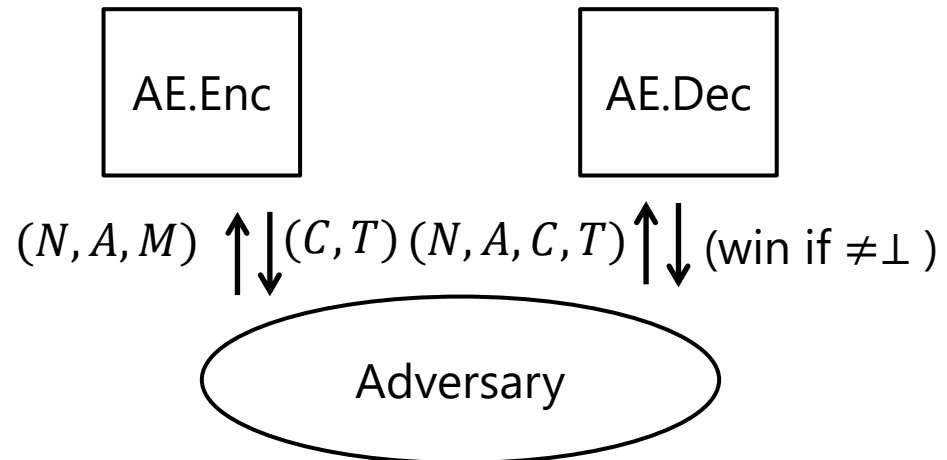
Security measures

- ◆ Defined by the games
- ◆ PRIV (Confidentiality) game: Adv tries to distinguish (C, T) from random
- ◆ AUTH game: Adv tries to create a non-trivial forgery of (N, A, C, T)

Game PRIV

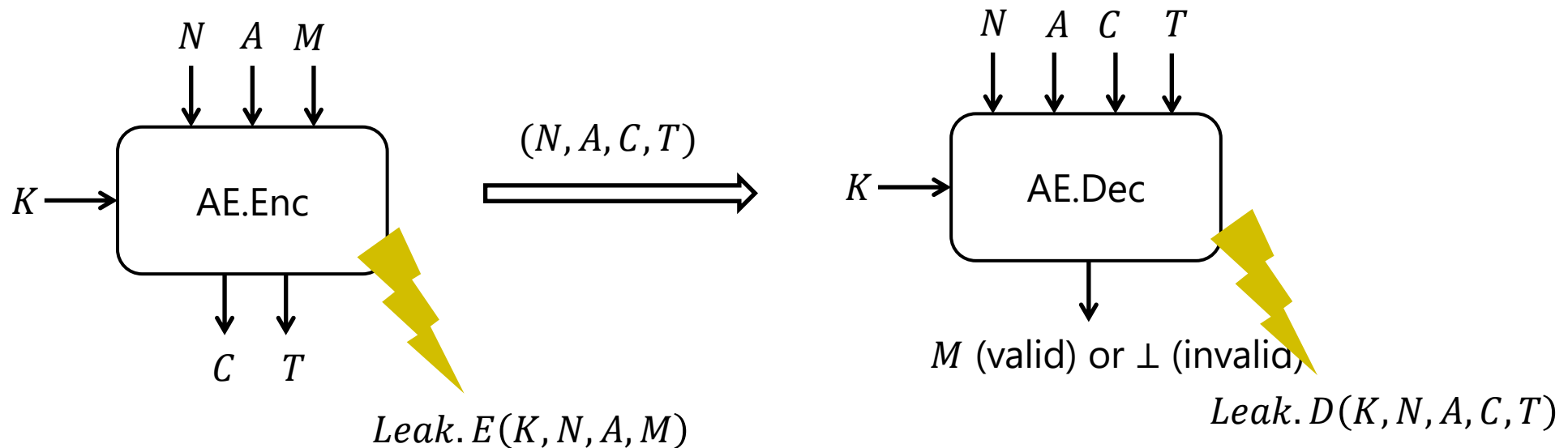


Game AUTH



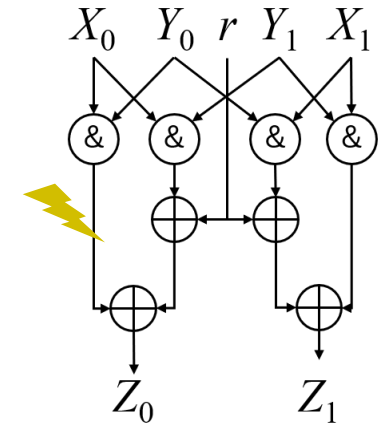
Background: Physical leakage

- ◆ Physical information on computation may leak: attacker can use (N, A, C, T) and $Leak.E / D(K, N, A, M)$, for example:
- ◆ Side-channel attacks (SCAs)
 - Timing: cache-timing, microarchitectural attack (e.g. Spectre)
 - Power consumption: differential power analysis (DPA), simple power analysis (SPA)
- ◆ Generic protection method (e.g. masking, const-time impl.) studied extensively for >25 years



Generic protection method: masking

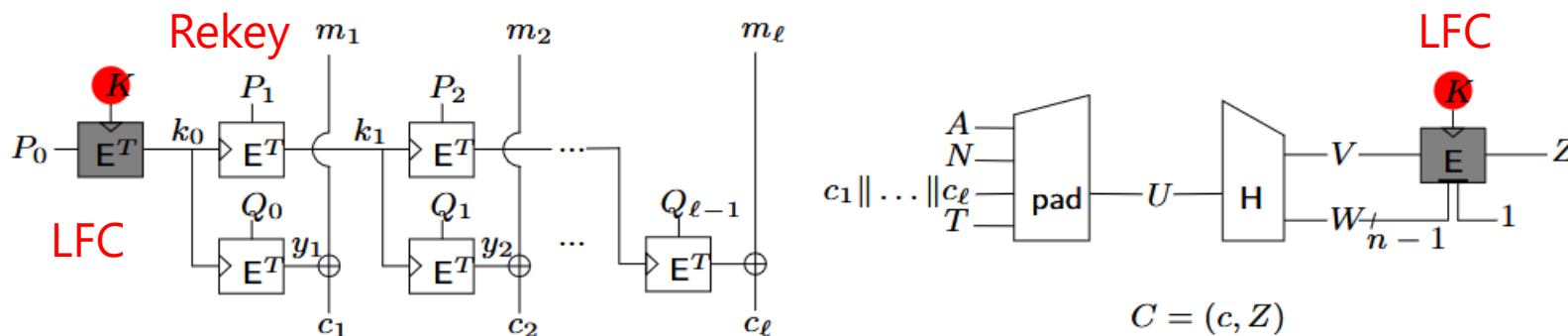
- ◆ Attacker puts d probes on the circuit
- ◆ d -th order masking (d -bounded protection)
 - Transform the circuit to keep functionality, but probing gives no information on input and output (secret sharing)
 - ISW circuit
- ◆ Cost: $O(d^2)$
- ◆ Generic solution of LRAE: masking the entire AE circuit (too costly!)
- ◆ We want more efficient methods
- ◆ Leakage-resilient AE (LRAE) = AE scheme with SCA protection by design



masked AND [GMK16]

Example: TEDT (Berti-Guo-Pereira-Peters-Standaert TCHES 2020)

- ◆ An LRAE based on a tweakable BC
- ◆ Enc-then-MAC, where
- ◆ Enc = Nonce-based initial key derivation + CTR-like mode by rekeying
- ◆ MAC = hash-then-TBC



TEDT <https://tches.iacr.org/index.php/TCHES/article/view/8400/7784>

Features of TEDT

◆ Two features common to many LRAEs:

■ Leak-Free Component, LFC (gray box)

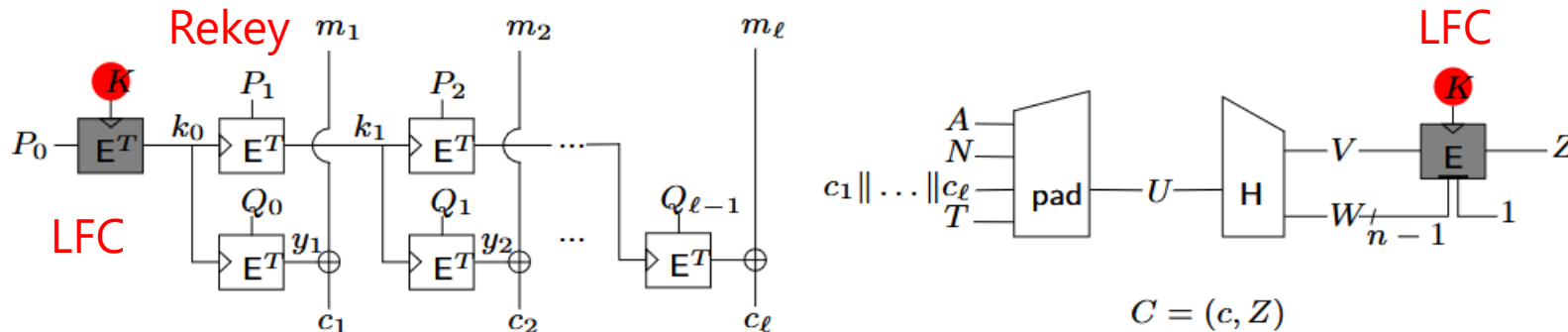
- a.k.a. "DPA-secure", "physically protected" etc.
- No matter what computation is done, LFC does not leak

■ Rekeying per message block

- By SPA-secure component (white box)
 - SPA-secure = no leak as long as key is new

◆ LFC is heavy, SPA-sec. component is light

◆ Leveled implementation: combined use of LFC + SPA-sec component, minimize total cost

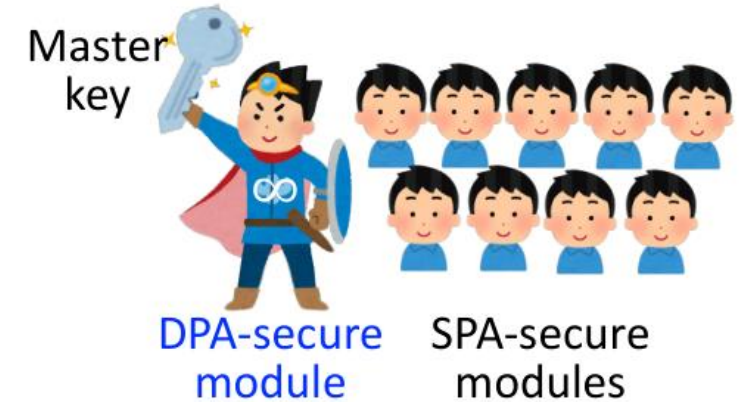


TEDT

<https://tches.iacr.org/index.php/TCHES/article/view/8400/7784>

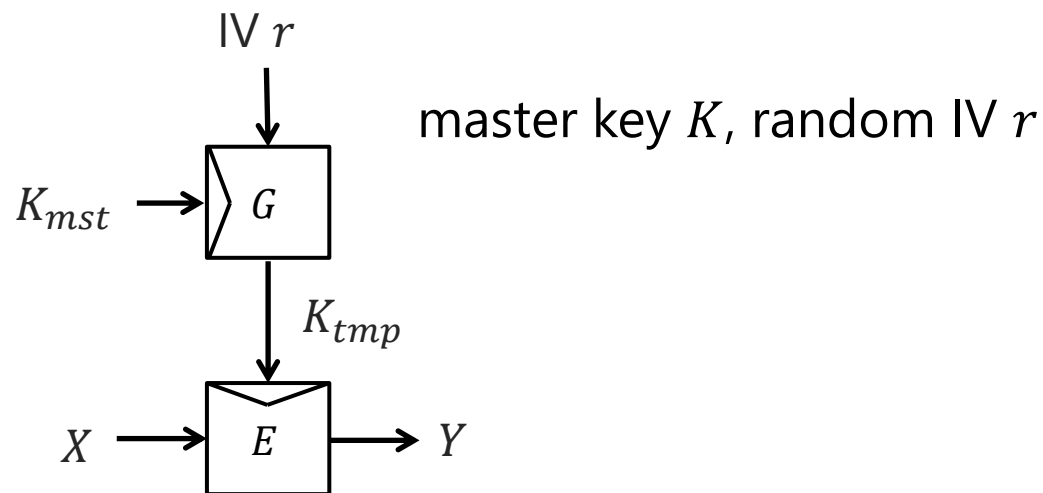
Research directions

- ◆ LF component is hard to achieve in practice
 - Difficulty of high-order masking
 - Advancement of SCAs, e.g. guided by ML
- ◆ Debates on practicality of LFC
- ◆ My opinion: both directions would be relevant (for research)
 - Boundary is not very strict, changing...
- ◆ So, first part: effort to **LFC-free** constructions (real world)
- ◆ Then the second part: **boosting efficiency** admitting LFC (ideal world)



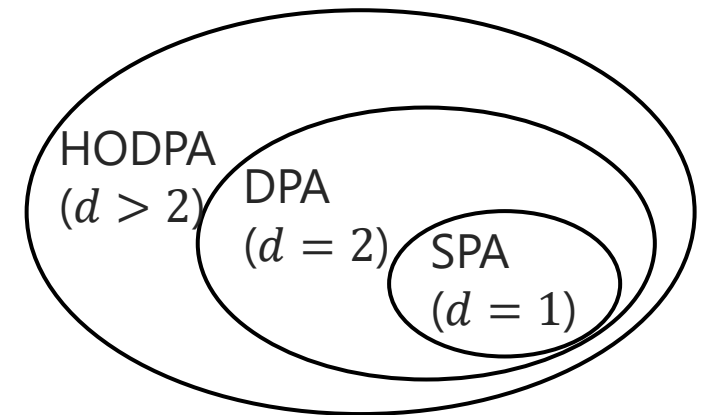
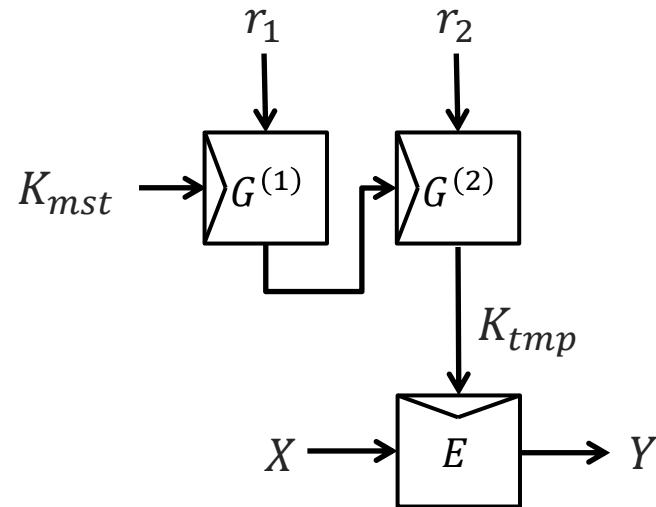
Fresh re-keying: A classical yet influential approach

- ◆ Goal: BC evaluation protected against DPA (or other high order attacks)
- ◆ Initial idea: DPA is not applicable when keys are changing
- ◆ Encryption by BC E with key K_{tmp} derived by KDF G
- ◆ Intuition: if G is DPA-secure (=LFC) and E is SPA-secure, $FR_K[G, E]$ is DPA-secure
 - SPA is much easier to protect
- ◆ Proposed by [MSGR10] and many follow-ups



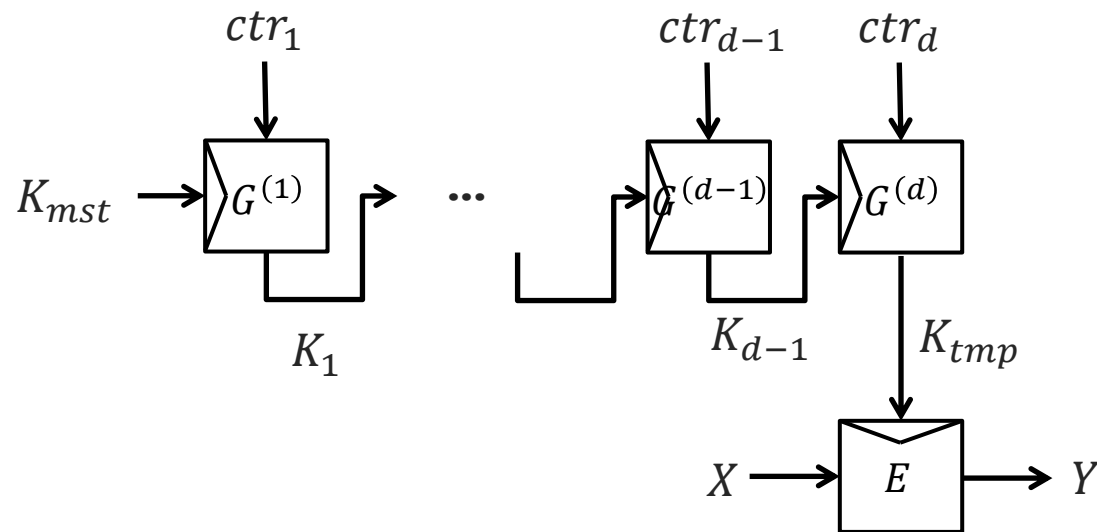
Practicality of FR

- ◆ “KDF is (only) DPA-secure” is not enough: broken by (more powerful) higher order attack
 - How/When can we limit the ability of adversary? Sometimes reasonable but not universal
- ◆ Desired goal is practically **unbounded** SCA security → efficient mechanism for gaining protecting order
- ◆ Cascading KDFs does not work: the root one still needs unbounded security



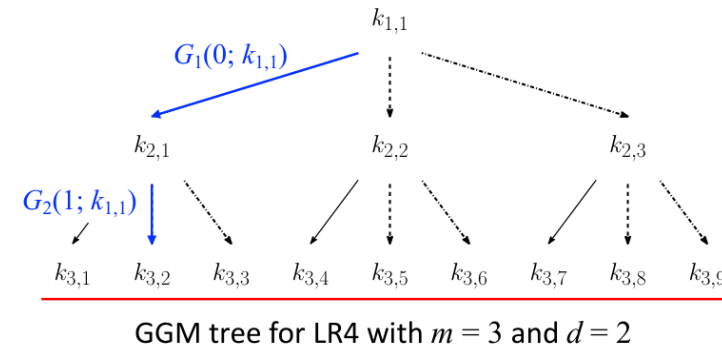
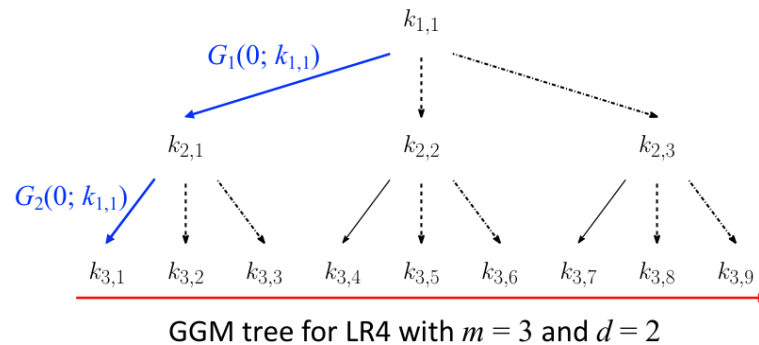
Solution: LR4 (Ueno-Homma-Inoue-Minematsu TCHES 2024)

- ◆ Problem of cascading KDFs: root one needs to run q times for q encryptions (and q may be too large to q -th order protection)
- ◆ But if we use counters instead of random IVs, the situation changes
 - We cache the temporal keys K_i for $i = 1, \dots, d$ (assumption: memory does not leak)
 - counters incremented from lower (ctr_d) to upper (ctr_1)
- ◆ Then each KDF $G^{(i)}$ needs to run only if $G^{(i-1)}$ runs $\rightarrow$ each runs $m = 2^{|ctr_i|}$ times

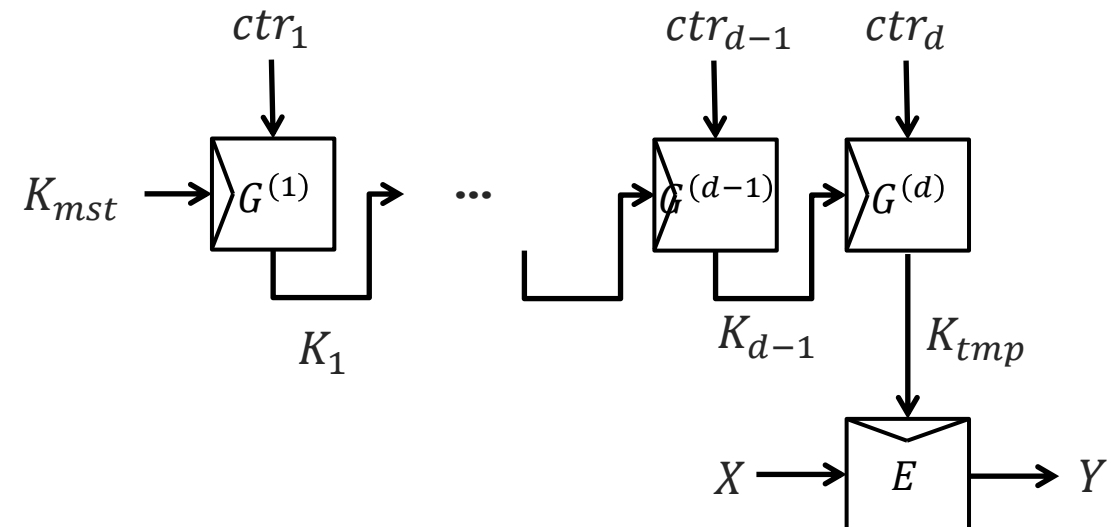


Tree evaluation

- ◆ Essentially, sequential evaluation of GGM tree
 - GGM tree was also used by LR-PRFs, but for different purposes



- ◆ By applying m -bounded protection to $G^{(i)}$, LR4 can encrypt $q = m^d$ times, so **exponential runs enabled by constant-protected modules**
- ◆ Average enc. cost: $< 2 \text{ KDFs} + 1 \text{ E}$



Provable security under leakage

- ◆ Dedicated security notion (similar to LAE notion but dedicated leakage function)
- ◆ Entire LR4 = TBC with counter tweak. # of enc/dec with a tweak is limited (protection of E)
- ◆ LR4-L : LR-4 encryption/decryption with leakage
 - stores the list of temporal keys and the number of its usage
 - If $\exists$ tmp. key with more than m usage (**overuse**), it will be leaked. Otherwise $\perp$
 - Captures m -bounded KDFs
- ◆ Assume independent RO for each $G^{(i)}$ (as in many LRAEs)
- ◆ Distinguishing game
 - Real: $(\text{LR4}^\pm, G = (G^{(1)}, \dots, G^{(d)}), \text{LR4-L}^\pm)$
 - Ideal: (Ideal TBC-E/D, $G = (G^{(1)}, \dots, G^{(d)}), \text{LR4-leak-E/D}$)
- ◆ Additional query rules to prevent trivial wins (forwarding)

$$\text{Adv}_{\text{LR4}}^{\text{LFR}}(\mathcal{A}) := \left| \Pr \left[\mathcal{A}^{\text{LR4}^\pm, G, E^\pm, \text{LR4-L}^\pm} \rightarrow 1 \right] - \Pr \left[\mathcal{A}^{\tilde{\text{P}}^\pm, G, E^\pm, \text{LR4-L}^\pm} \rightarrow 1 \right] \right|,$$

Security result

- ◆ Overused key → key collision, hence allowing traditional collision-based analysis
 - Also, any key collision (before overuse) breaks black-box security
 - Thus, mostly identical to BB security proof: nontrivial leak = bad event in BB security

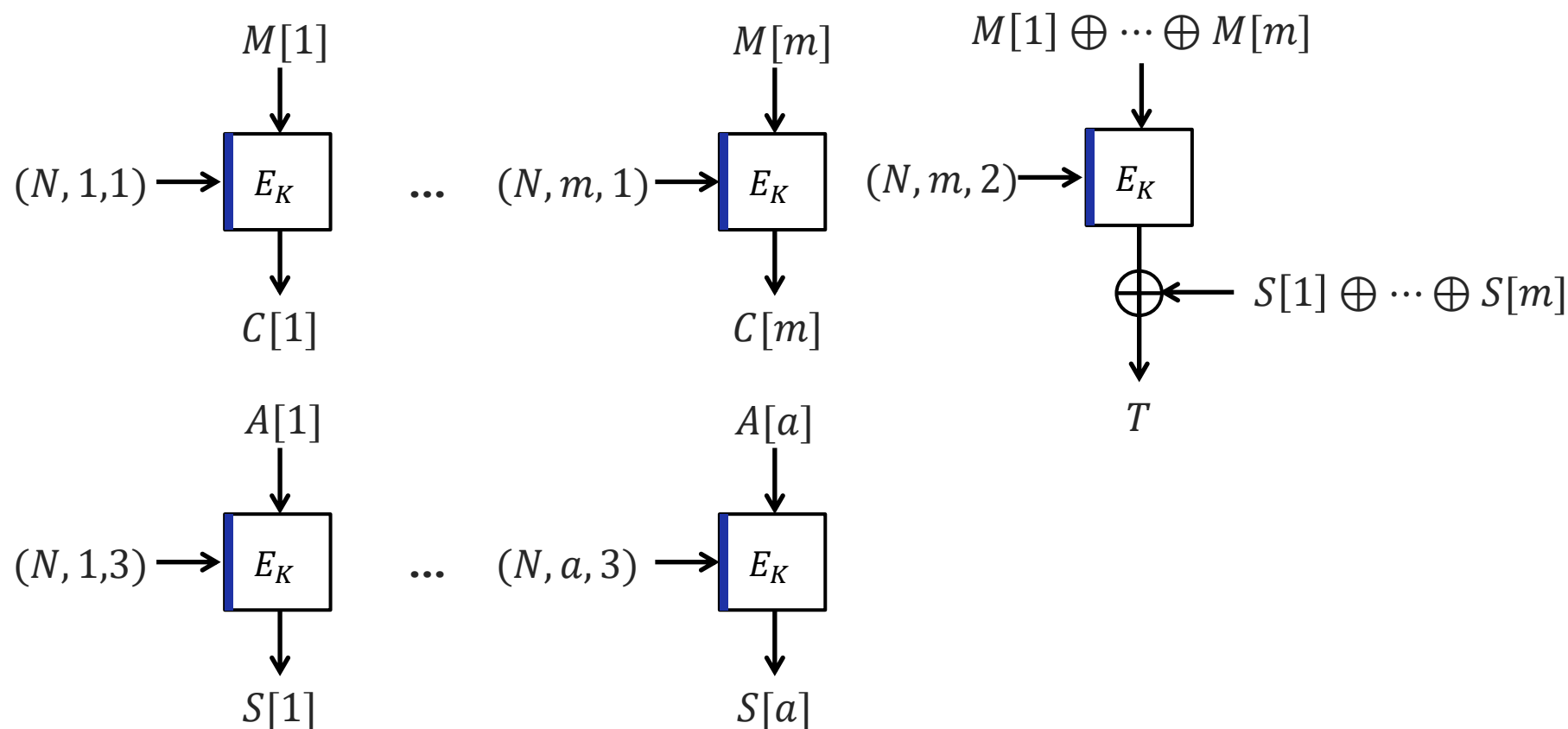
Theorem 1. *Let $\mathcal{A}$ be the adversary following LFR game. Let q be the total number of construction queries, and q_L be the total number of leakage queries. For $i \in [d]$, we assume $\mathcal{A}$ queries p_i times to G_i and p_I times to E . Then, we have*

$$\text{Adv}_{\text{LR4}}^{\text{LFR}}(\mathcal{A}) \leq \frac{d(q + q_L)^2}{2^{n_k+1}} + \frac{(q + q_L)(p + p_I)}{2^{n_k}} + \frac{4m'q}{2^{n_{bc}}},$$

where $p = \sum_{i=1}^d p_i$ and $q \leq 2^{n_{bc}-1}$. We also assume $q_L \leq m'2^{n_{ctr}d}$.

Applications of LR4

- ◆ Challenge-response
- ◆ OCB-style parallel AE w/ FR → LRAE for encryption-leakage (Mennink AC20)
- ◆ LR4 could also be used, given the tweak sequence is implemented by an up-counter incl. N

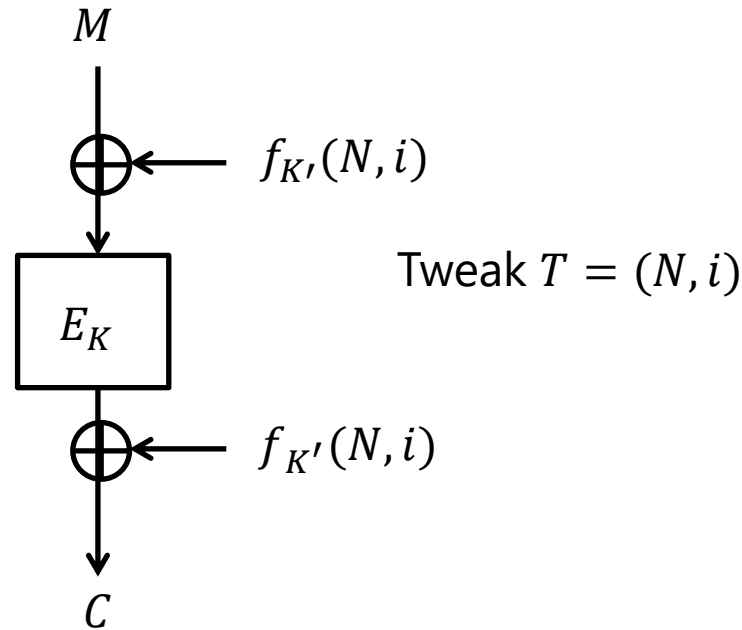


Second part: the ideal world (LFC is available)

- ◆ Many LFC-based designs such as FGHF', TEDT, with different levels of protections
- ◆ (Per block, per nonce) Rekeying is costly. Can we remove?
- ◆ Recent work (Ueno-Inoue-Minematsu-Ito-Homma Crypto 2026): main contribution is a formal security proof of masking
- ◆ As a side-product, we propose efficient LRAE [w/o rekeying](#)

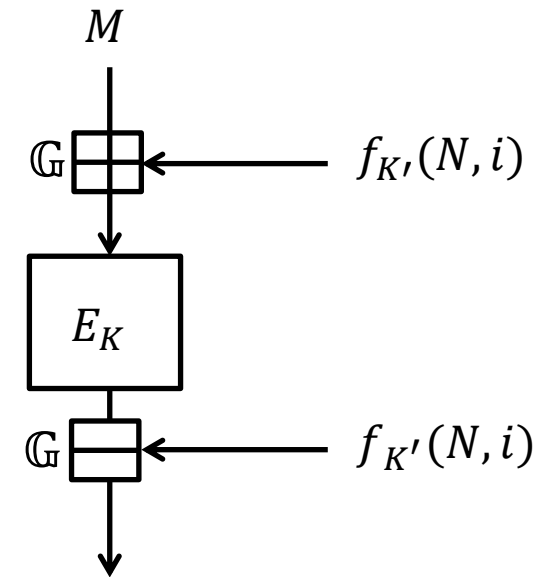
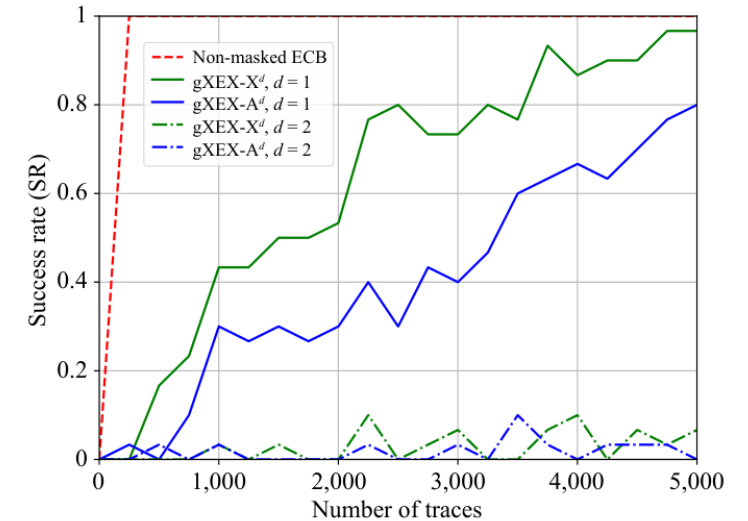
XEX (Rogaway AC04)

- ◆ XOR-Encrypt-XOR to implement TBC
- ◆ Building block of numerous schemes
- ◆ Offset/mask generation is ϵ -Almost-XOR-Universal (AXU)
 - $\Pr[f_i(L) \oplus f_j(L) = \delta] \leq \epsilon$ for any $i \neq j, \delta \in GF(2^n)$
 - Doubling: $f_{K'}(N, i) = 2^i L$ for $L = E_{K'}(N)$
 - Gray code: $f_{K'}(N, i) = i$ -th word of GC for the basis $\{2^j L\}_{j=0,1,\dots}$
- ◆ Offset provides intrinsic 1st-order masking scheme under non-repeating tweak [UIT+25]



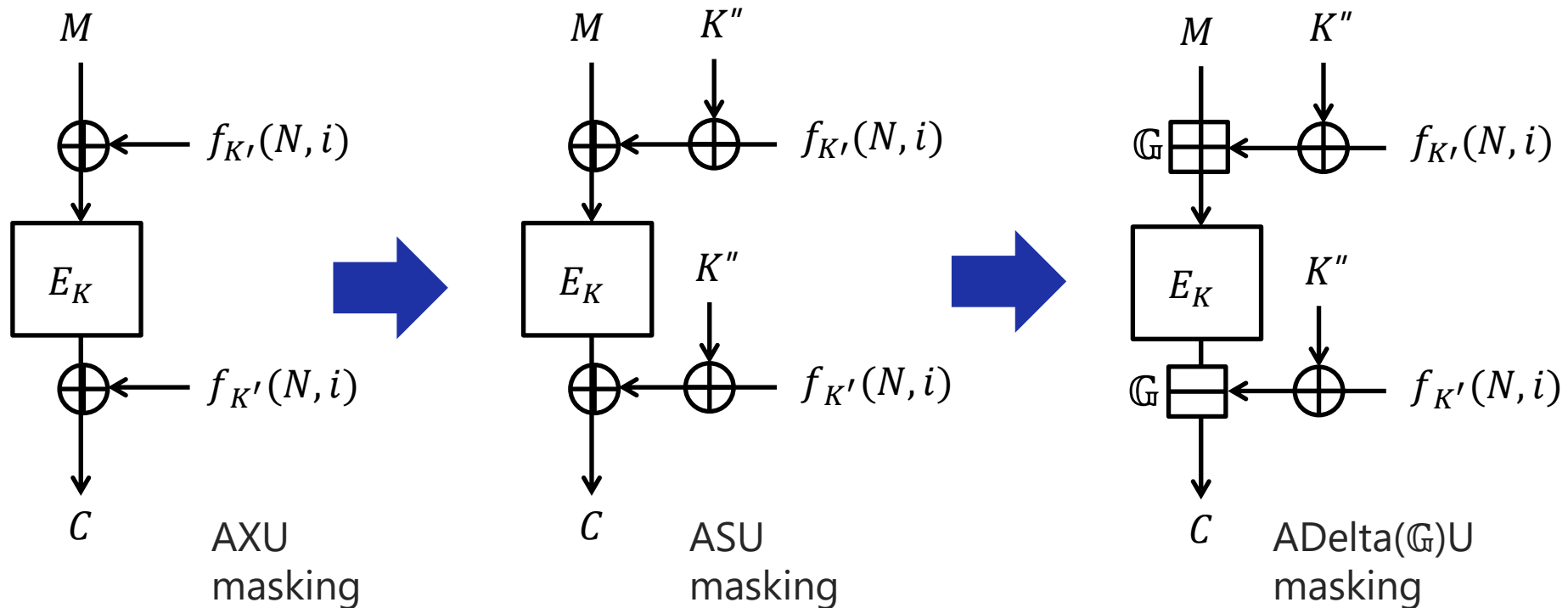
Generalizing XEX

- ◆ [UEM+26] shows the mask addition group $\mathbb{G}$ greatly affects security
- ◆ XOR is weak (Boolean masking)
- ◆ $\text{mod } 2^n$ is strong (Arithmetic masking)
- ◆ Depending on the environment, desirable $\mathbb{G}$ may change
 - Platform
 - E.g., $\mathbb{G} = (\mathbb{Z}/2^{64}\mathbb{Z}, \mathbb{Z}/2^{64}\mathbb{Z})$ may be suitable for 64-bit platform
 - Desired security level (# of traces and attack order etc.)
- ◆ BB security needs Almost-Delta($\mathbb{G}$)-universal, but
- ◆ Using traditional doubling/GC does not guarantee black-box security if $\mathbb{G} \neq GF(2^n)$
 - i.e. $AXU \rightarrow A\Delta(\mathbb{G})U$ does not hold
 - needs ad-hoc analysis... inconvenient



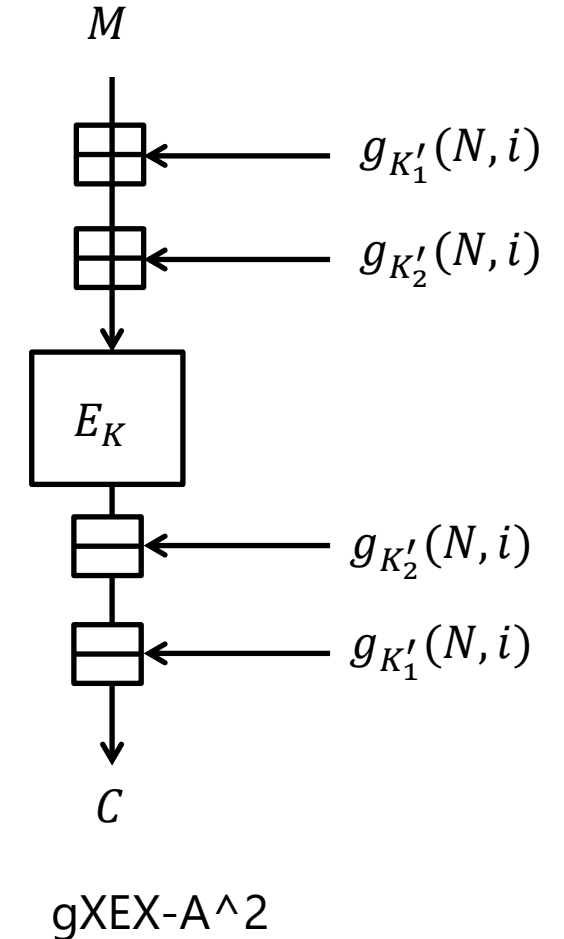
Generalizing XEX (gXEX[$E, \mathbb{G}$])

- ◆ Transform the known AXU function by a second key:
- ◆ AXU + second key $\rightarrow$ ASU (Almost strongly uniform) $\rightarrow$ ADelta($\mathbb{G}$)U for any abelian $\mathbb{G}$
- ◆ Classical, but in a new context



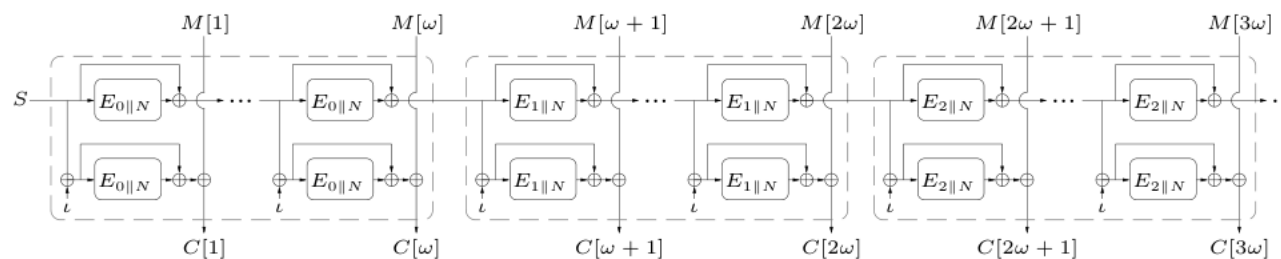
gXEX-A^d and AOCB

- ◆ Use d independent masking (keys may be derived by KDF)
- ◆ Call it gXEX-A^d (as special form of gXEX[$E, \mathbb{G}$]) with $\mathbb{G} = \mathbb{Z}/2^{(128)}\mathbb{Z}$
- ◆ Suggested implementation strategy:
 - LFC: initial evaluation of $g_{K'_i}(N, 0)$ for $i = 1, \dots, d$
 - Plain: mask incrementation (derive $g_{K'_i}(N, j)$ from $g_{K'_i}(N, j - 1)$), addition/subtraction
 - Plain: mainline E_K
 - May need additional care for other types of SCA
- ◆ AOCB: OCB-like mode built on gXEX-A^d
- ◆ AOCB has formal security of d -bounded protection as an AE
 - For encryption-leakage
 - No rekey, rate-1 $\rightarrow$ the best LRAE efficiency we can hope
 - Different protection level from existing LRAEs



UEDT: LRAE built on single-length key BC

- ◆ (Guo-Khairallah-Minematsu TCHES 26): another effort to improve the efficiency of TEDT
- ◆ TEDT and others rely on double-length-key BC, e.g. AES-256, less efficient than single-length-key BC, AES-128
- ◆ Revising TEDT Enc and MAC parts so that it allows to use single-length-key BC
 - Known construct (LR-BC-3[BMPS21]) does not present formal BB security and rate 1/5
- ◆ UEDT: rate 1/4 and $\sim n$ -bit BB security + $n/2$ -bit leak security (as TEDT)
 - Enabled by chunk-wise fixed-key encryption inspired by CENC, utilizing MJH hash, etc.



Encryption of UEDT (PSVDM)

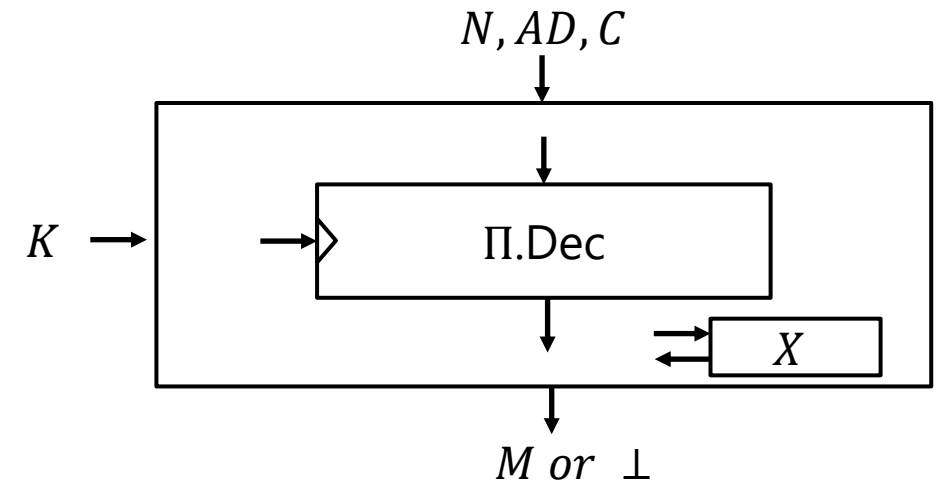
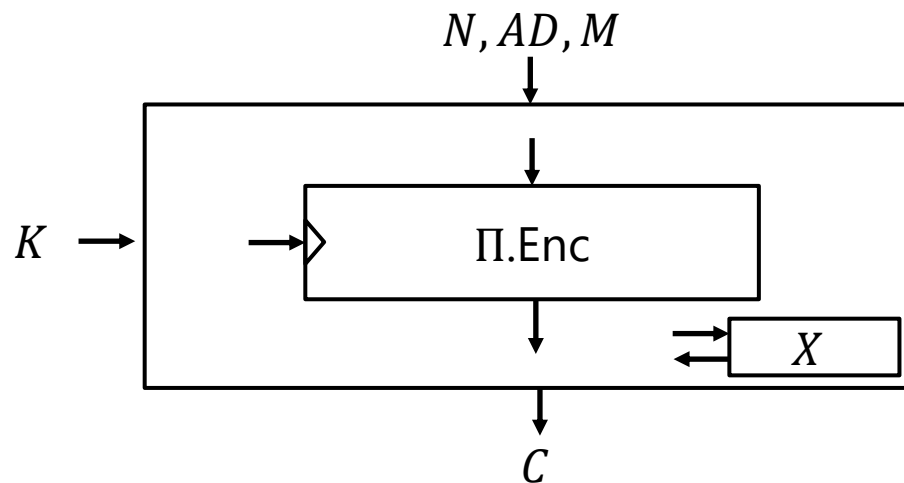
Committing Authenticated Encryption

Background (committing security)

- ◆ Recent hot topic: committing (CMT) security of AE
- ◆ Hardness of finding a valid input tuple **including keys** given a valid ciphertext C
 - $(K, N, A, M) \neq (K', N', A', M')$ so that $\Pi(K, N, A, M) = \Pi(K', N', A', M') = C$
- ◆ Classes of CMT sec. notions
 - CMT-1 (Key committing): Imposing $K \neq K'$
 - CMT-4 (Context committing): no restriction, stronger notion than CMT-1 (our focus)
- ◆ Known AEs typically do not satisfy CMT, because they are not designed so
- ◆ Lack of CMT security leads to several real-world problems
 - Message franking by Facebook, Partitioning oracle attack, SFrame attack and many more

Generic Transform

- ◆ Generic transform: $GT [\Pi, X]$ for additional component(s) X
 - All elements of X are typically keyless, e.g. a hash function
- ◆ Security goal of GT: AE security and CMT security for $GT [\Pi, X]$
 - AE security = PRIV (IND-CPA) security + AUTH (INT-CTXT) security
- ◆ Provable security reductions to AE security of Π (PRIV, AUTH) and some assumptions on X

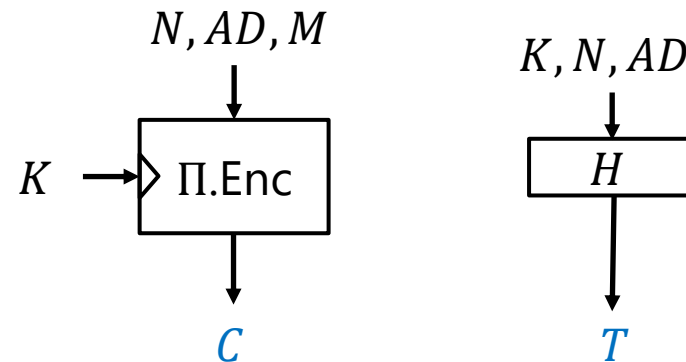


Chen and Karadžić, Eurocrypt 2025

- ◆ Three basic generic transforms: HtAE, AEaH, EtH
- ◆ Two advanced generic transforms: AEtH and chaSIV
- ◆ Features:
 - **Strictly generic**
 - Preserving (Nonce-based) AE security of Π
 - MRAE preservation for advanced schemes
 - Proving CMT-4 and CDY (context discovery: out of our focus)
 - Main ingredient: hash function $H: \{0,1\}^* \rightarrow \{0,1\}^n$

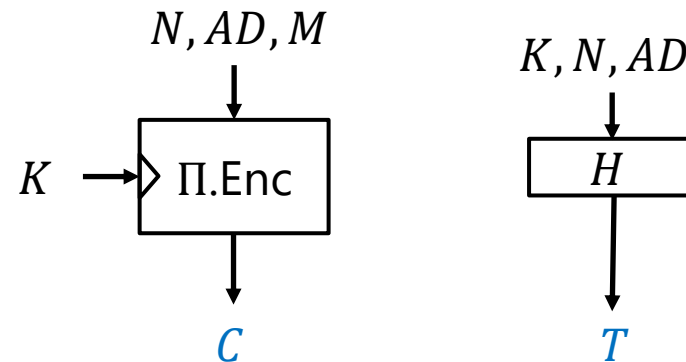
AE and Hash (AEaH) [CK25]

- ◆ AE Π takes (K, N, ε, M) , output C
- ◆ Hash H takes (K, N, AD) , output T
- ◆ Total output: (N, AD, C, T)
 - Output gets longer, a cost to pay make it generic
- ◆ Decryption: perform AE decryption and hash check, accept if both OK



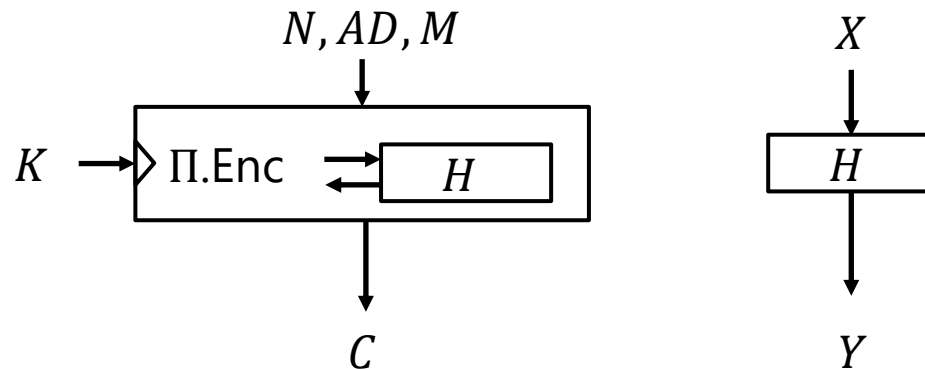
Security claims of AEaH

- ◆ PRIV/AUTH bounds are proved assuming $H=RO$, $\Pi = \text{secure AE}$
- ◆ CMT bound is proved assuming $H=CR$ hash
- ◆ Other CK25 schemes use similar assumptions



Mixing two security models

- ◆ **A mix** of standard and idealized models. Theoretically fine (we know what it means), but how to instantiate?
- ◆ H would be instantiated by a concrete algorithm such as SHA-2/3
- ◆ **What if this instantiated H is also used by Π ?**
 - Out of the assumption -- RO is not instantiable in principle
- ◆ But practical: GT must be independent of AE, but AE **can** depend on (the algorithm of) GT
 - Malicious/backdoored AE



PRIV (IND-CPA) attack against AEaH

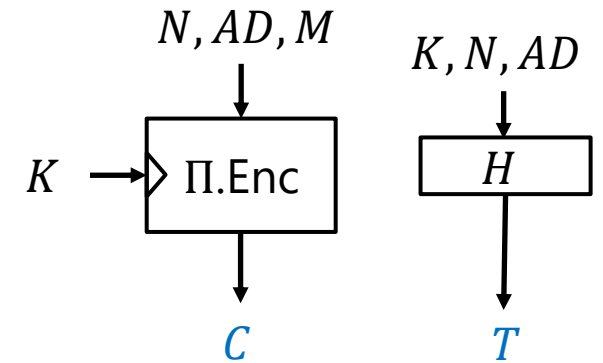
◆ Let $\text{AE.Enc} = \text{Enc-then-MAC}$

◆ $\text{Enc} = \text{CTR w/ } H(K, N, \text{ctr})$

■ $\text{ctr} = 1, 2, \dots$

■ easily realize a secure EtM-AE assuming $H(K, *) = \text{PRF}$

■ MAC can reuse $H(K, \cdot)$ with a domain separation



PRIV attack

1. Encrypt (N, AD, M) w/ $AD = "1"$. $M = M[1]M[2] \dots$

2. Receives (C, T)

3. Checks if $C[1] \oplus T = M[1]$. If so "real", otherwise "random"

In the real world: $(C, T) = \text{AEaH}(K, N, AD, M)$, $C[1] = M[1] \oplus H(K, N, 1)$, $T = H(K, N, 1)$.

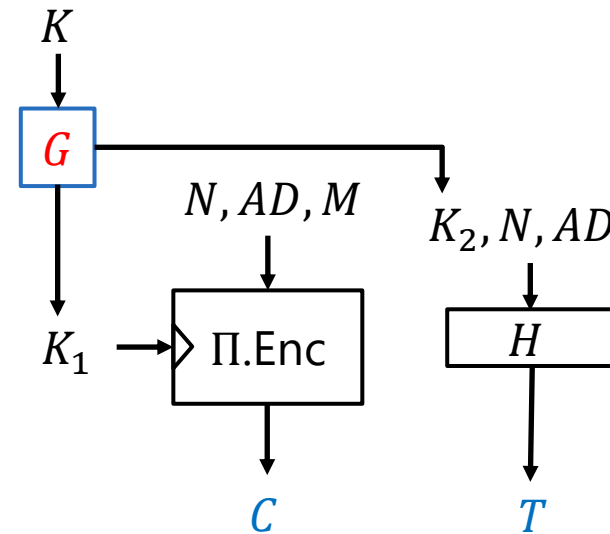
→ PRIV break

- When M is unknown, this implies (partial) plaintext recovery

- A slight variation allows key recovery

Fix: Key separation to remove ideal primitive

- ◆ Apply $G: \mathcal{K} \rightarrow \mathcal{K}^2$ for key separation on the top: one for AE, one for GT
- ◆ Rekey but one-time: avoiding (costly) nonce-based rekeying
- ◆ Establish entirely STM proofs of the modified transforms: now safely instantiable
 - assuming right-half-CR [BH22] and PRG for G and PRF for H



Summary of our results (Inoue-Minematsu, ToSC 2026 (to appear))

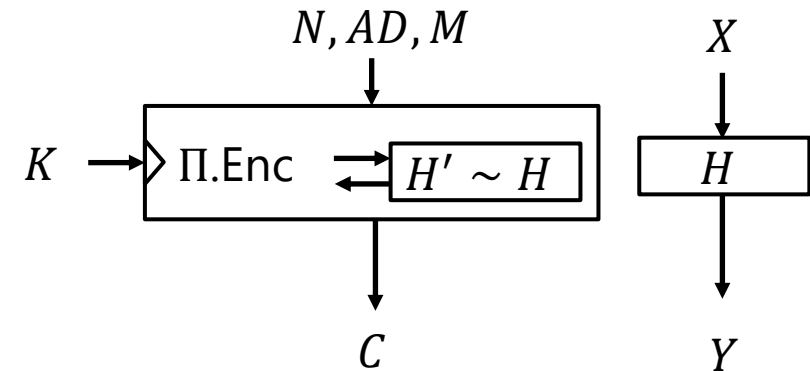
Scheme	Strictly generic?	Nonce-based Rekey	Sec. Model	Priv ATK	Auth ATK	Key recov.
CTX [CR22]	N		Mixed	?	?	?
CTY [BH24]	N		Mixed	?	?	?
HtE-UtC/RtC [BH22]	Y	Y	Standard			
HtAE [CK25]	Y	Y	Mixed	?	?	?
AEaH [CK25]	Y		Mixed	Y	Y	(Y)
EtH [CK25]	Y		Mixed	Y	Y	(Y)
AEtH [CK25]	Y		Mixed	Y	?	(Y)
chaSIV [CK25]	Y	Y	Mixed	eY	?	(eY)
PACT [BBD24]	~Y		Mixed	Y	Y	(Y)
comPACT [BBD24]	~Y		Mixed	Y	Y	(Y)

~Y: API is generic, AE needs addit. condition.

(Y): attackable w/ slightly artificial AE inst.
eY: attackable w/ some GT insts.

Interpretation of the attacks

- ◆ Response: "too trivial/known?"
- ◆ → But we could not find the **same** attack situation in the literature
- ◆ Response: "They are caused by the (wrongly) shared ideal primitives, so instantiation does not matter?"
- ◆ → The presented attacks can be interpreted so, but
 - AE was assumed to be in STD model, so basically ideal primitives should not be inside
 - Could not capture the situation if AE uses a modified version of instantiation of H
 - AES-based hashing vs. 9-round AES-based PRF/MAC
 - Full SHA-3 hash vs Keccak-p-based PRF
- ◆ Response: "Can we resolve?"
- ◆ → Beyond STM-only reconstruction, not sure
- ◆ Opportunities for future research??



Thank you!